

AI-Assisted Formalization of Discrete Mathematics

Entrance Assessment

Prof. Wang Qiufeng · Stud. Wang Hongwei ·

School of Advanced Technology & School of Mathematics and Physics, XJTLU · Duration: 1 Month

Qiufeng.Wang@xjtlu.edu.cn · Hongwei.Wang22@student.xjtlu.edu.cn

Overview

This assessment is the entry point into the **AI-Assisted Formalization of Discrete Mathematics** project. Our long-term goal is to formalize high-school and undergraduate combinatorics in Lean 4 and use the resulting dataset to train AI models capable of translating and solving discrete-mathematics problems automatically.

This assessment is **not** designed to filter out people who already know Lean 4. It is designed to identify participants who are genuinely motivated to learn. Curiosity and persistence matter far more than prior experience. **You are encouraged to use any large language model (ChatGPT, Claude, etc.) throughout.**

■ *There is no penalty for using AI assistants. The skill we evaluate is your ability to translate mathematical ideas into Lean 4 types — not your ability to memorize syntax.*

Assessment Structure

Part	Task	Weight
Part 1	Installation guide & learning resources	—
Part 2	Formalize 8 combinatorial statements	50 pts
Part 3	Formally prove 5 theorems	50 pts

Submission: a single `.lean` file named `YourName_Assessment.lean`. Online submissions via `live.lean-lang.org` are also accepted.

Part 1 — Getting Started with Lean 4 (Reference Guide, Not Graded)

This part is a **guide, not a graded task**. It walks you through setting up Lean 4 and points to resources you will need. You do not need a local installation to submit — the online playground works perfectly fine.

Option A — Online Playground (Recommended for Beginners)

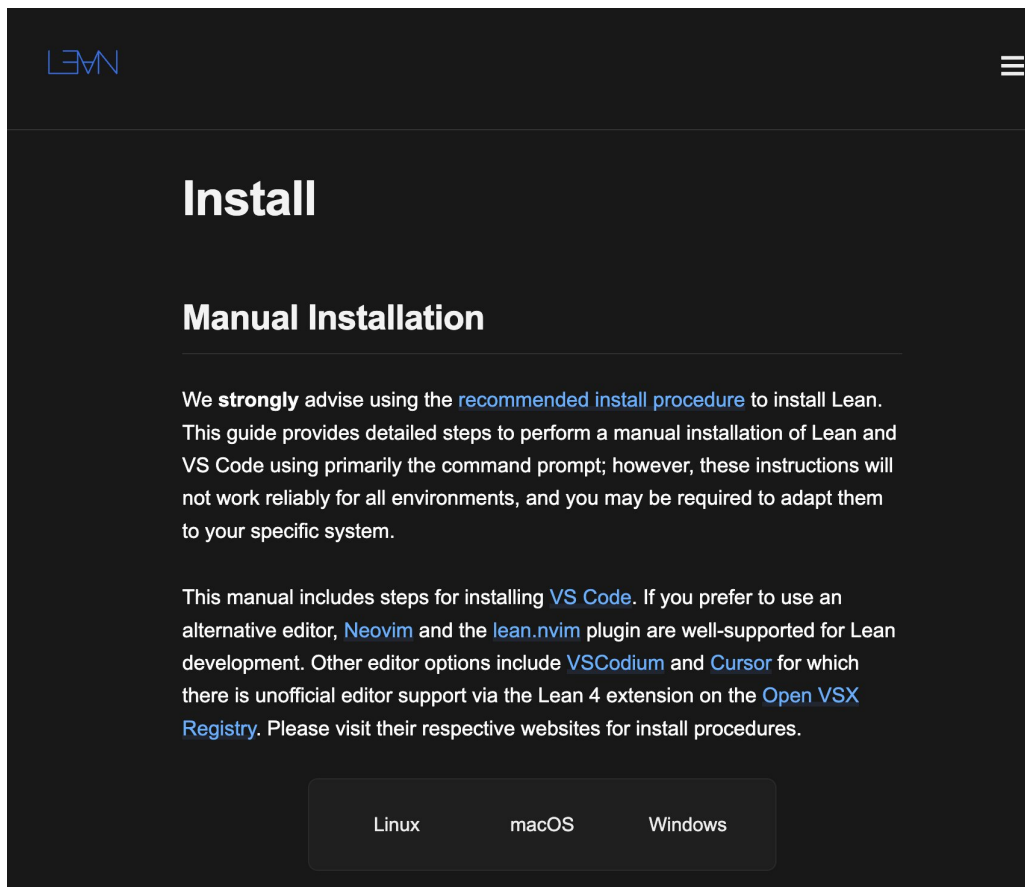
The easiest way to start is the official Lean 4 web editor. No installation required.

<https://live.lean-lang.org>

Open the link, clear the example code, paste your solution, and wait for the blue spinner to finish. A green checkmark means your proof compiles. Share your code directly in your .lean file.

Option B — Local Installation (VS Code + Lean 4 Extension)

For a full local setup, visit the official installer page shown below:



lean-lang.org/install/manual/ — choose Linux, macOS, or Windows

Step 1 — Install elan (the Lean version manager)

Open a terminal and run:

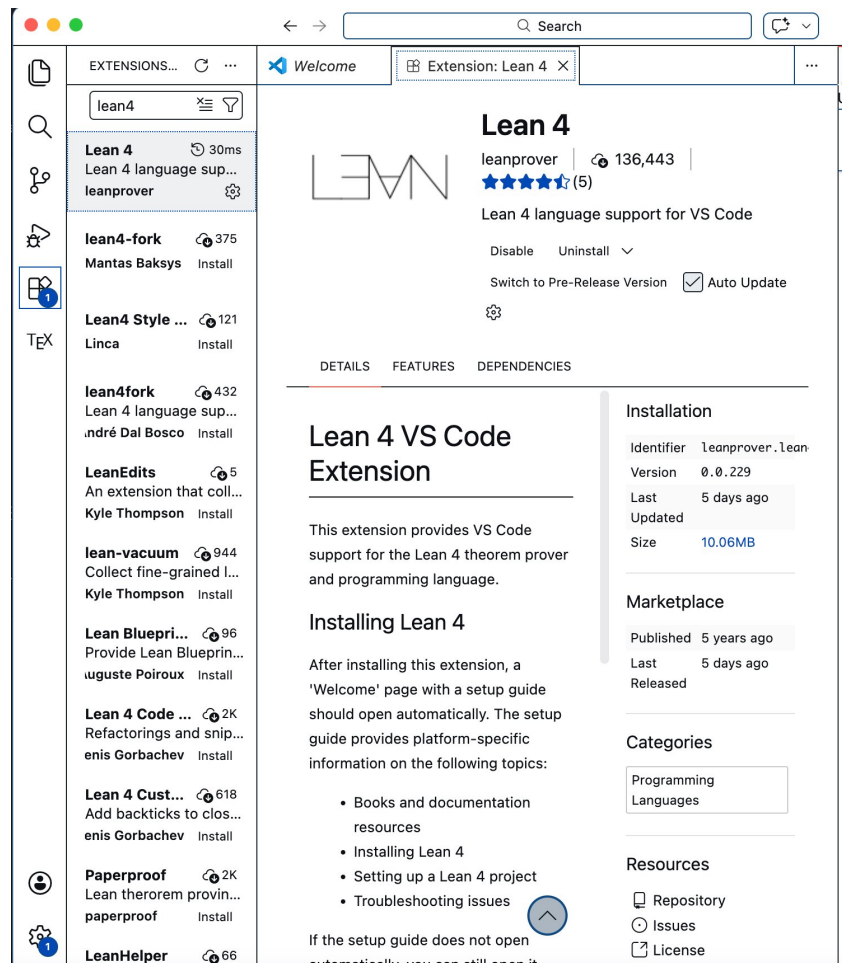
```
# macOS / Linux
curl https://elan.lean.dev/elan-init.sh -sSf | sh

# Windows (PowerShell, run as Administrator)
curl -O https://elan.lean.dev/elan-init.exe; ./elan-init.exe

# Restart terminal, then verify:
elan --version
```

Step 2 — Install VS Code and the Lean 4 extension

Download VS Code from <https://code.visualstudio.com/>, open the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X), search for **lean4**, and install the extension by *leanprover*:



VS Code Extensions panel — search 'lean4' and install the leanprover extension

Step 3 — Create a new project and fetch Mathlib

```
lake new MyAssessment
cd MyAssessment
lake exe cache get # downloads Mathlib — may take 5-15 min
code .
```

■■ The first time Mathlib loads in VS Code it may appear frozen for several minutes. This is normal — it is building the index. Do not close the editor.

Step 4 — Import Mathlib in your file

Add this line at the top of any .lean file where you need Mathlib definitions:

```
import Mathlib
```

Learning Resources

- **Introductory article in Chinese**

<https://zhuanlan.zhihu.com/p/678954230>

- **Natural Number Game — learn proof tactics by playing**

<https://adam.math.hhu.de/#/g/leanprover-community/nng4>

- **Official Lean 4 installer**

<https://lean-lang.org/install/manual/>

- **Online Lean 4 playground**

<https://live.lean-lang.org/>

- **Mathlib4 documentation**

https://leanprover-community.github.io/mathlib4_docs/Mathlib

- **Lean community home**

<https://leanprover-community.github.io/>

- **Lean community chat (Zulip) — ask any question, beginners very welcome**

<https://leanprover.zulipchat.com/>

- **Theorem Proving in Lean 4 (official textbook)**

https://leanprover.github.io/theorem_proving_in_lean4/

- **Mathematics in Lean — Chinese translation**

<https://www.leanprover.cn/math-in-lean-zh/>

As of 31 December 2025, Mathlib4 contains 123,189 mathematical definitions and 253,478 theorems, covering essentially all undergraduate mathematics and many graduate topics. Think of it as a Noah's Ark for mathematics — rigorously verified, one theorem at a time.

Part 2 — Formalizing Statements (50 pts, ~6 pts each)

For each problem, write a valid Lean 4 **type signature** that faithfully captures the mathematical statement. **Use `sorry` as the proof body — no proof required.** Problems increase in difficulty: 2.1–2.3 are straightforward; 2.4–2.6 require more careful use of Mathlib types; 2.7–2.8 are more challenging.

Tip: use Mathlib search or ask an LLM to find the right types. Key names: `Nat.factorial`, `Nat.choose`, `Finset`, `Fintype`, `Function.Injective`, `Finset.card`, `SimpleGraph`.

Problem 2.1 — Factorial is Positive

For every natural number n , $n! > 0$.

Hint: `Nat.factorial_pos`.

```
-- Problem 2.1: Factorial is Positive
theorem problem_2_1 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.2 — Factorial Growth

For all natural numbers $n \geq 1$, $n! \geq n$.

Hint: search for `Nat.factorial` in `Mathlib`. You may express $n \geq 1$ as a hypothesis $h : 1 \leq n$.

```
-- Problem 2.2: Factorial Growth
theorem problem_2_2 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.3 — Symmetry of Binomial Coefficients

For all natural numbers n and k with $k \leq n$, $C(n, k) = C(n, n - k)$.

Hint: `Nat.choose_symm`.

```
-- Problem 2.3: Symmetry of Binomial Coefficients
theorem problem_2_3 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.4 — Pascal's Identity — Statement Only

For all natural numbers n and k , $C(n+1, k+1) = C(n, k) + C(n, k+1)$.

Hint: `Nat.choose_succ_succ`.

```
-- Problem 2.4: Pascal's Identity – Statement Only
theorem problem_2_4 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.5 — Pigeonhole Principle

If $f : \text{Fin } (n+1) \rightarrow \text{Fin } n$ is any function, then f is not injective.

Hint: try `Function.not_injective_infinite_finite` or `Fintype.exists_ne_map_eq_of_card_lt`.

```
-- Problem 2.5: Pigeonhole Principle
theorem problem_2_5 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.6 — Inclusion-Exclusion for Two Finite Sets

For any two finite sets A and B , $|A \cup B| + |A \cap B| = |A| + |B|$. Note: use the addition form to avoid subtraction on natural numbers.

Hint: `Finset.card_union_add_card_inter`.

```
-- Problem 2.6: Inclusion-Exclusion for Two Finite Sets
theorem problem_2_6 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.7 — Vandermonde's Identity

For all natural numbers m, n, r : the sum over k from 0 to r of $C(m, k) * C(n, r-k)$ equals $C(m+n, r)$.

Hint: Look for `Nat.add_choose_eq` in `Mathlib`, or state it as an equality of `Finset.sum` and `Nat.choose`.

```
-- Problem 2.7: Vandermonde's Identity
theorem problem_2_7 : <YOUR TYPE HERE> := by
  sorry
```

Problem 2.8 — Handshake Lemma — Statement

In any simple finite graph G on vertex type V , the sum of all vertex degrees equals $2 * |E|$. State this using SimpleGraph from Mathlib.

Hint: SimpleGraph.sum_degrees_eq_twice_card_edges. Import Mathlib.Combinatorics.SimpleGraph.Degree.

```
-- Problem 2.8: Handshake Lemma – Statement
theorem problem_2_8 : <YOUR TYPE HERE> := by
  sorry
```

Part 3 — Formal Proofs (50 pts)

Provide complete Lean 4 proofs for the five theorems below. Problems 3.1–3.3 are high-school combinatorics; 3.4–3.5 touch university-level discrete mathematics. **Partial proofs are welcome** — use sorry for steps you are stuck on and add a comment explaining your approach.

Allowed tactics: rfl, simp, omega, induction, cases, exact, apply, intro, constructor, use, have, linarith, norm_num, decide, ring, and any Mathlib lemma. Interactive search tactics exact?, apply?, simp? are also encouraged.

Problem 3.1 — Gauss Sum (8 pts)

Prove that $2 * (0 + 1 + 2 + \dots + n) = n * (n + 1)$.

Use induction on n , or invoke Finset.sum_range_id directly.

```
-- Problem 3.1: Gauss Sum
theorem gauss_sum (n : ℕ) : 2 * ∑ i in Finset.range (n + 1), i = n * (n + 1) := by
  sorry
```

Problem 3.2 — Pascal's Identity (8 pts)

Prove: for all $n, k : \mathbb{N}$, $C(n+1, k+1) = C(n, k) + C(n, k+1)$.

Nat.choose_succ_succ states this exactly — one line suffices. For a more instructive proof, use cases on k and unfold Nat.choose.

```
-- Problem 3.2: Pascal's Identity
theorem pascal_identity (n k : ℕ) :
  Nat.choose (n + 1) (k + 1) = Nat.choose n k + Nat.choose n (k + 1) := by
  sorry
```

Problem 3.3 — Sum of Binomial Coefficients (10 pts)

Prove that the sum of the n -th row of Pascal's triangle equals 2^n . That is: for all $n : \mathbb{N}$, $(\sum_{k=0}^n C(n,k)) = 2^n$.

Look for Nat.sum_range_choose in Mathlib — it may give this directly. Alternatively, prove by induction using Pascal's identity.

```
-- Problem 3.3: Sum of binomial coefficients
theorem binom_row_sum (n : ℕ) :
  ∑ k in Finset.range (n + 1), Nat.choose n k = 2 ^ n := by
  sorry
```

Problem 3.4 — Pigeonhole (Constructive) (12 pts)

Let $f : \text{Fin } (n+2) \rightarrow \text{Fin } (n+1)$. Prove there exist distinct $i \neq j$ with $f(i) = f(j)$.

Use `Fintype.exists_ne_map_eq_of_card_lt`. You need `card (Fin (n+2)) > card (Fin (n+1))`, which `simp` or `omega` closes.

```
-- Problem 3.4: Pigeonhole Principle (constructive)
theorem pigeonhole (n : ℕ) (f : Fin (n + 2) → Fin (n + 1)) :
  ∃ i j : Fin (n + 2), i ≠ j ∧ f i = f j := by
  sorry
```

Problem 3.5 — Handshake Lemma (12 pts)

Let G be a simple finite graph. Prove that the sum of all vertex degrees equals $2 * |E|$.

`SimpleGraph.sum_degrees_eq_twice_card_edges` is in `Mathlib` — one line suffices. For a from-scratch proof, observe each edge $\{u,v\}$ contributes 1 to $\text{deg}(u)$ and 1 to $\text{deg}(v)$.

```
-- Problem 3.5: Handshake Lemma
theorem handshake_lemma {V : Type*} [Fintype V] [DecidableEq V]
  (G : SimpleGraph V) [DecidableRel G.Adj] :
  ∑ v, G.degree v = 2 * G.edgeFinset.card := by
  sorry
```

Submission Guidelines

- Submit a **single .lean file** named `YourName_Assessment.lean`.
- Structure the file with `-- ## Part N` section comments.
- Add a short English comment above each answer explaining your reasoning.
- Online submissions via **live.lean-lang.org** are fully accepted.
- You may freely use LLMs, Mathlib search, Zulip community chat, and any other resource.
- If Lean refuses to compile something, include the error message as a comment — showing you tried matters more than a perfect result.
- Partial credit is given for well-structured attempts with `sorry` and clear explanations.

This seminar is informal — please ask questions freely in the group chat and discuss anything, related or unrelated. The formalization of mathematics is about making ideas precise enough that a machine can verify them. The journey is just as important as the destination.

We look forward to working with you. Good luck! ■